# Detecção de quedas com sensores de movimento: limiares × aprendizado de máquina

Quanto cada método perde ao ser levado para outra base de dados (e para o celular no bolso)?
Relatório do experimento, Projeto Mova (UNAMA), outubro de 2026

**Resumo em uma frase.** A rede neural (CNN-LSTM, reimplementada de Duan et al. 2026) é a melhor *dentro* da base em que foi treinada (F1 de 0,93 a 0,99, com 99,0% de acurácia por janela, igual ao artigo), mas é a que mais perde ao ir para outra base. Da cintura (SisFall) para o celular no bolso (MobiFall), o F1 caiu para 0,21–0,56. O melhor algoritmo de limiar (Kangas 2008) quase não perdeu: F1 0,95 → 0,89. Porém, os limiares que dependem de picos de impacto altos (Bourke3 e PIPTO) desabaram no celular, porque o acelerômetro do celular da MobiFall satura em ±2 g. Ou seja, a robustez não depende de ser "limiar" ou "IA", e sim de o método depender ou não de características do sensor (faixa de medida e orientação).

## Resumo


**Pergunta.** Quando um detector de quedas é ajustado ou treinado numa base de dados e testado em outra, os algoritmos de limiar perdem desempenho tanto quanto um modelo leve de aprendizado de máquina (TinyML)?

**O que foi feito.** Implementamos em Python três algoritmos de limiar publicados (Kangas 2008, Bourke 2010 e PIPTO 2023; o PIPTO roda com o *código original dos autores*) e reimplementamos a CNN-LSTM de 21.434 parâmetros de Duan et al. (2026). Todos foram avaliados com o mesmo pré-processamento e a mesma regra de pontuação em duas bases públicas:

- **SisFall:** sensor na cintura, 38 pessoas (15 idosas), 4.396 tentativas;
- **MobiFall v2.0:** smartphone no bolso da calça, 24 pessoas, 630 tentativas.

Testamos as duas direções (SisFall → MobiFall e MobiFall → SisFall). Em cada uma, o detector é ajustado ou treinado numa base e aplicado na outra sem nenhum retreino. O processamento levou cerca de 4 horas em CPU.

**Resultados principais (SisFall → MobiFall, a direção que interessa ao Mova):**

1. **Dentro da base, a CNN-LSTM é a melhor.** F1 = 0,989 (regra k=1) e 0,934 (regra k=2 do artigo) na SisFall. A acurácia por janela foi de 99,0%, praticamente a mesma do artigo original (99,3% na KFall). A reimplementação reproduz o resultado "de laboratório".
2. **Fora da base, a CNN-LSTM é a que mais perde.** Na MobiFall, o F1 caiu para 0,564 (k=1) e 0,211 (k=2): ΔF1 = 0,43 e 0,72. Com a normalização usada no artigo ("z-score por base"), o F1 foi 0,000.
3. **O melhor limiar quase não perde.** Kangas com os parâmetros publicados: F1 0,947 → 0,891 (ΔF1 = 0,056, IC 95% 0,012 a 0,100). Reajustado na SisFall: 0,964 → 0,924. Os intervalos de confiança do Kangas e da CNN-LSTM não se sobrepõem.
4. **Mas nem todo limiar é robusto.** Bourke3 (F1 0,755 → 0,163) e PIPTO (0,816 → 0,086) desabaram no celular. A causa foi isolada experimentalmente: o acelerômetro do Galaxy S3 usado na MobiFall satura em ±2 g, então quase nenhum pico de queda passa dos limiares de impacto deles (2,8 g e ≈3,06 g). Ao simular essa saturação na própria SisFall, a sensibilidade do PIPTO caiu de 81% para 36% e a do Bourke3 de 61% para 47%. A do Kangas ficou em 93%.
5. **Parte da perda da CNN vem da orientação do sensor.** A gravidade aparece no eixo y com sinal oposto nas duas bases. Treinar a CNN com rotações 3D aleatórias reduziu a perda pela metade (ΔF1 0,43 → 0,21), mas não eliminou.
6. **A direção inversa confirma.** Da MobiFall para a SisFall, a CNN-LSTM (k=1) caiu de 0,926 para 0,609, com 149 alarmes falsos por hora. O Kangas reajustado na MobiFall foi de 0,924 para 0,967, ou seja, não piorou.

**Resposta à pergunta.** Para o melhor algoritmo de limiar (Kangas), a hipótese H1 se confirma: o modelo de aprendizado de máquina perde **muito mais** ao trocar de base. Isso **não** vale para "limiares

em geral". Algoritmos que dependem de picos altos de impacto são tão ou mais frágeis que a rede neural quando o sensor muda. O resultado também contradiz, nas nossas condições, a perda de apenas ~1% relatada por Duan et al. (2026) na transferência KFall → SisFall. A diferença provável é que a nossa troca de base envolve mudança de *posição* e de *aparelho* (cintura → bolso), e não apenas de pessoas.

**Roteiro de 1 minuto para explicar ao professor.** (1) *Pergunta:* algoritmos de limiar perdem tanto quanto IA ao mudar de base? (2) *Por que esses algoritmos:* Kangas e Bourke3 são os únicos limiares já reavaliados de forma independente em quedas reais (Bagalà 2012); o PIPTO é o limiar recente (2023) com código público; a CNN-LSTM é o TinyML mais recente (2026), que afirma generalizar bem. (3) *Por que essas bases:* são públicas; a SisFall é a mais usada da área e tem idosos; a MobiFall é celular real no bolso, o caso do Mova. (4) *Como medimos:* ajusta ou treina numa base e testa na outra sem mexer em nada; mesma pontuação para todos; separação por pessoa; intervalos de confiança por bootstrap. (5) *Resultado:* a IA é a melhor dentro da base e a que mais perde fora dela; o limiar certo (Kangas) quase não perde; limiares que dependem de picos altos quebram com o sensor do celular. (6) *Por que importa:* mostra que validar só na mesma base engana, e indica o que o Mova deve usar como linha de base.

# 1. Contexto e pergunta de pesquisa

O Mova é um aplicativo (Expo / React Native) que usa o acelerômetro do celular para detectar quedas de idosos e avisar cuidadores. O detector caseiro da primeira versão errava muito. Em vez de inventar um modelo novo, a equipe decidiu agir como pesquisadora: pegar algoritmos já publicados e medir, de forma controlada, se eles continuam funcionando fora das condições em que foram criados.

A literatura mostra que isso é um problema real. Bagalà et al. (2012) testaram 13 algoritmos publicados em quedas reais de idosos: a sensibilidade média foi de 57% e houve de 3 a 85 alarmes falsos por dia, muito pior do que os artigos originais diziam. Revisões recentes (2024–2026) apontam que quase todos os estudos usam quedas simuladas e testam na mesma base em que treinaram. O artigo TinyML de Duan et al. (2026) afirma que algoritmos de limiar "generalizam mal", mas não testa nenhum limiar no seu protocolo. Este experimento faz esse teste.

## Hipóteses

- **H1:** o aprendizado de máquina vai melhor dentro da base de treino, mas perde mais desempenho ao mudar de base; os limiares são mais estáveis.
- **H0:** a perda (Δ) é igual nos dois tipos de método.
- **H1-alternativa:** o aprendizado de máquina continua melhor e perde pouco (o que Duan et al. afirmam: ≈1%).

# 2. O que mudou em relação ao plano, e por quê

| Plano original | O que foi feito | Motivo |
|---|---|---|
| Treinar na KFall e testar na SisFall (protocolo de Duan et al.) | Treinar na SisFall e testar na MobiFall, e o inverso | A KFall exige pedido de acesso e o site dela estava bloqueado no ambiente de execução. A MobiFall (celular real no bolso) estava acessível e é **mais relevante para o Mova**, porque testa a ida da cintura para o celular. |
| LOSO (deixar 1 sujeito de fora, 32 vezes) | Validação cruzada por sujeito em 5 partes | O LOSO completo multiplicaria o tempo de treino por 7 a 8 (estimativa de mais de 12 horas em CPU). Os 5 folds por sujeito mantêm a regra essencial (nenhum sujeito no treino e no teste ao mesmo tempo). |
| Só limiares com parâmetros publicados | Também limiares reajustados na base de treino | A CNN é treinada na base de treino. Para a comparação ser justa, os limiares também foram reajustados nela. |
| — | Experimentos de controle: saturação do sensor, rotação 3D, faixa ampliada de limiares, taxa de 200 Hz | Para explicar por que cada método caiu, e não só medir quanto caiu. |

# 3. Materiais

## 3.1 Bases de dados

| | SisFall | MobiFall v2.0 |
|---|---|---|
| Referência | Sucerquia, López & Vargas-Bonilla, *Sensors* 17(1):198, 2017 | Vavoulas et al., IJMSTR 2014 (BMI Lab, Creta) |
| Sensor e posição | dispositivo próprio (ADXL345, ±16 g) preso na **cintura** | **smartphone** Samsung Galaxy S3 no **bolso da calça** (acelerômetro ±2 g) |
| Taxa original | 200 Hz, regular | ≈90 Hz, irregular (com carimbo de tempo em ns) |
| Pessoas | 23 jovens (19–30 anos) + 15 idosos (60–75); só 1 idoso (SE06) fez quedas | 24 adultos de 22 a 47 anos; ADLs de 9 deles |
| Tentativas usadas | 4.396 (1.744 quedas; 2.652 ADLs; 14,2 h de ADL) | 630 (288 quedas; 342 ADLs; 2,5 h de ADL) |
| Tipos | 15 quedas (F01–F15), 19 ADLs (D01–D19) | 4 quedas (FOL, FKL, BSC, SDL), 9 ADLs |
| Onde obtivemos | espelho no GitHub (krishna-karthikk/SisFall-Dataset; 4.398 dos 4.505 arquivos) | espelho no GitHub (yehowlong/MobiAct_Dataset_v2.0-MobiFall_Dataset_v2.0) |

"ADL" (*activities of daily living*) são as atividades normais: andar, sentar, subir escada, pular etc. Elas servem para medir os **falsos positivos**.

## 3.2 Como os algoritmos funcionam (o sinal não é "um número só")

O acelerômetro mede a aceleração nos 3 eixos (x, y, z) dezenas de vezes por segundo. Neste estudo usamos 50 vezes por segundo, então cada tentativa de 15 s vira uma série de 750 × 3 valores. Os três algoritmos de limiar procuram o **formato** típico de uma queda nessa série:

1. **Queda livre:** a aceleração total $|a| = \sqrt{(x^2+y^2+z^2)}$ cai bem abaixo de 1 g por uma fração de segundo;
2. **Impacto:** logo depois aparece um pico alto;
3. **Pós-queda:** a pessoa fica deitada (a gravidade passa a apontar para o lado do corpo) ou parada.

A magnitude |a| junta os 3 eixos num número por instante, mas a dimensão do tempo continua. A etapa de postura usa o vetor 3D da gravidade.

| Algoritmo | Regra (parâmetros publicados) | Fonte da implementação |
|---|---|---|
| **Kangas et al. 2008** (variante 2) | Queda livre: $\|a\| < 0{,}6$ g. Impacto em até 1 s, se *qualquer* um destes for verdadeiro: $\|a\| \geq 2{,}0$ g; $\|a$ sem gravidade$\| \geq 1{,}7$ g; variação em 0,1 s ≥ 2,0 g; aceleração vertical estimada ≥ 1,5 g. Postura 2 s depois: componente vertical ≤ 0,5 g (deitado). | Bagalà et al. 2012 (texto completo) e o porte aberto "Guardian" (licença MIT) |
| **Bourke et al. 2010** ("Bourke3") | Limiar inferior 0,65 g e de impacto 2,8 g. Tempo de descida ≤ 600 ms e de subida ≤ 350 ms. Velocidade vertical ≤ −0,7 m/s. Postura entre 1 e 3 s depois: inclinação > 60° em mais de 75% do tempo. | Bagalà et al. 2012 (texto completo) |
| **PIPTO** (Moutsis et al. 2023) | Pares "baixo" ($\|a\| < 6{,}5$ m/s²) → "alto" ($\|a\| >$ máx(média, 20) + 10 m/s², ou seja ≥ ≈3,06 g) em até 1,05 s, com até 0,85 s de duração. Depois, o sinal precisa ficar perto de 1 g e parado. | **Código original dos autores** (GitHub), chamado sem modificação |

## 3.3 A rede neural: CNN-LSTM de Duan et al. (2026)

A entrada são janelas de 4 s (200 amostras a 50 Hz) com 4 canais: ax, ay, az e a magnitude. Cada canal passa por uma convolução própria. Os resultados são combinados por outra convolução e depois por duas camadas LSTM (memória de curto e longo prazo) e três camadas densas. O total é de 21.434 parâmetros, conferido no código.

**Correção encontrada no artigo.** A Tabela 1 de Duan et al. informa núcleo (8,1) na primeira convolução. Com esse núcleo, não dá para obter ao mesmo tempo a saída (24,1,16) e os 544 parâmetros que a própria tabela informa. O núcleo **(33,1) com passo 7** reproduz exatamente as dimensões, os parâmetros de cada camada e o total de 21.434. Usamos essa versão e recomendamos declarar isso no artigo.

Treino como no artigo:

- otimizador Adam com taxa de aprendizado 0,001, 25 épocas;
- pesos de classe 0,8 para ADL e 0,2 para queda (escolha intencional dos autores para reduzir falsos positivos);
- normalização z-score;
- janela rotulada como queda se cobre ≥ 50% do intervalo da queda.

Duas suposições nossas, porque o artigo não informa: lote de 64 e ativação ReLU. Uma tentativa é classificada como queda se houver ≥ k janelas positivas seguidas: k = 2 (regra do artigo) ou k = 1.

# 4. Métodos (o processo científico passo a passo)

## 4.1 Pré-processamento, igual para todos

1. Converter tudo para g com a gravidade incluída, e conferir que |a| ≈ 1 g com a pessoa parada: deu 0,997 g na SisFall e 1,009 g na MobiFall.
2. Reamostrar para 50 Hz com filtro anti-*aliasing*. A MobiFall, que é irregular, foi antes interpolada numa grade de 100 Hz.
3. Saturar em ±8 g, para simular um celular atual. Isso não altera nenhuma decisão, como mostra o controle da Seção 5.4.
4. **Intervalo da queda**, usado só para rotular as janelas de treino da CNN: [pico − 1,0 s, pico + 0,5 s]. A SisFall e a MobiFall não marcam o instante da queda.
5. **Referência de "em pé"** para a postura, por sujeito: a mediana da direção da gravidade no 1º segundo das tentativas.

## 4.2 "Antes do treino" e "depois do treino"

- **Limiares publicados = antes de qualquer treino.** Usam os números dos artigos e são aplicados direto nas duas bases. Mostram o desempenho "de prateleira".
- **Limiares ajustados:** uma grade de valores (35 combinações no Kangas e 21 no Bourke3) é testada na base de treino, e fica a de maior F1. O desempenho "dentro da base" vem de validação cruzada por sujeito.
- **CNN-LSTM:** o desempenho "dentro da base" vem de validação cruzada por sujeito em 5 partes. Cada pessoa é testada por um modelo que nunca a viu. O desempenho "entre bases" vem de 3 modelos treinados com *toda* a base de treino (sementes diferentes), aplicados à outra base sem retreino. Vale a decisão da maioria das 3 sementes.

## 4.3 Pontuação e métricas

- **Por tentativa (arquivo):** tentativa de queda com ≥ 1 alarme = acerto (verdadeiro positivo); tentativa de ADL com ≥ 1 alarme = falso positivo. É a mesma regra para todos, e a única possível na SisFall, que não marca o instante da queda.
- **Sensibilidade (SE):** fração das quedas detectadas. **Especificidade (SP):** fração das ADLs sem alarme. **F1:** média harmônica entre precisão e sensibilidade. **FA/h:** alarmes falsos por hora de atividade normal.
- **Δ (perda ao trocar de base)** = métrica na base de treino − métrica na outra base. Δ maior significa pior generalização.
- **Intervalo de confiança de 95% do Δ:** *bootstrap* de sujeitos. As pessoas são sorteadas com reposição 1.000 vezes, separadamente em cada base. Se o IC de um método não se sobrepõe ao de outro, a diferença entre eles é estatisticamente clara.

## 4.4 Experimentos de controle (para explicar as quedas de desempenho)

- **Saturação do sensor:** limiares publicados na SisFall com o sinal cortado em ±16 (original), ±8, ±4, ±3 e ±2 g por eixo.
- **Rotação 3D:** a CNN treinada com cópias das janelas giradas aleatoriamente em 3D, para ficar insensível à orientação do sensor.
- **z-score "por base" × "do treino":** o artigo normaliza cada base com as próprias estatísticas; a forma justa usa só as do treino.
- **Faixa ampliada de limiares:** alguns parâmetros escolhidos ficaram na borda da grade. Repetimos o ajuste com uma faixa maior (84 e 60 combinações).

- **Taxa original de 200 Hz:** limiares publicados na SisFall sem reamostrar, para medir o efeito da redução para 50 Hz.

### 4.5 Ambiente

Python 3.13, NumPy, SciPy, pandas, TensorFlow 2.21 (CPU), 4 núcleos, 16 GB de RAM. O experimento principal levou 3 h 17 min. Todo o código está no notebook `notebooks/estudo_limiar_vs_ml.ipynb`, que roda do início ao fim com um botão.

# 5. Resultados

## 5.1 Limiares "de prateleira" (sem treino) nas duas bases

| Algoritmo | SisFall SE | SisFall SP | SisFall F1 | SisFall FA/h | MobiFall SE | MobiFall SP | MobiFall F1 | MobiFall FA/h |
|---|---|---|---|---|---|---|---|---|
| Kangas | 93,5% | 97,4% | 0,947 | 4,8 | 89,6% | 90,4% | 0,891 | 13,1 |
| Bourke3 | 60,9% | 99,7% | 0,755 | 0,6 | 9,0% | 98,5% | 0,163 | 5,2 |
| PIPTO | 81,3% | 88,2% | 0,816 | 21,9 | 4,5% | 100,0% | 0,086 | 0,0 |

Na cintura (SisFall), o Kangas já é o melhor: SE 93,5%, SP 97,4% e 4,8 alarmes falsos por hora. O PIPTO detecta bem (81%), mas dispara muito: 22 alarmes por hora, principalmente em pulos (D19) e tropeços (D18). No celular no bolso (MobiFall), só o Kangas continua funcionando. O Bourke3 detecta 9% das quedas e o PIPTO 4,5%.

## 5.2 Da cintura para o bolso: SisFall → MobiFall

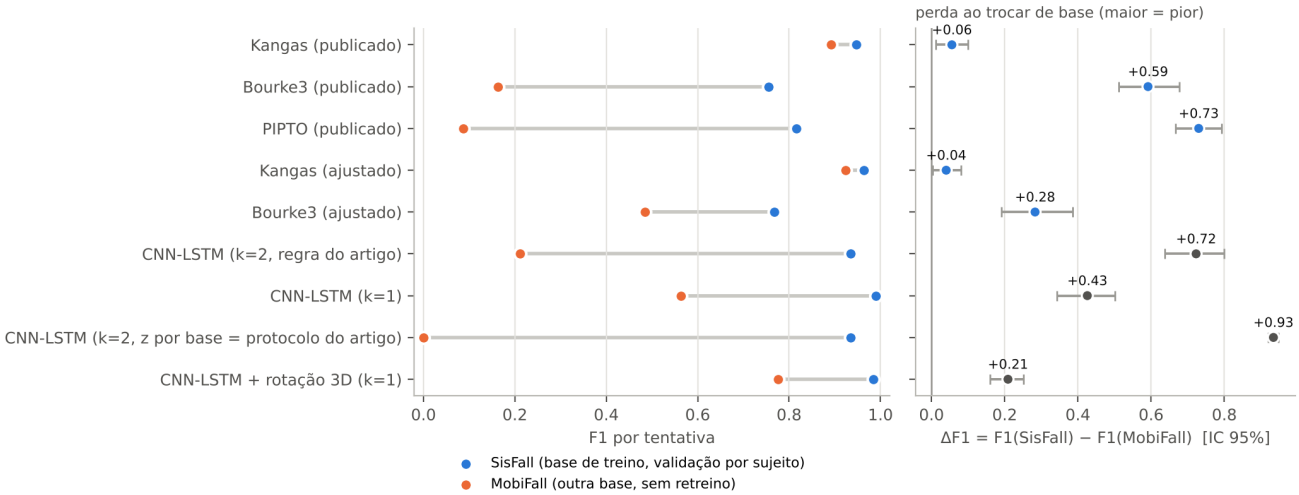



*Figura 1. F1 por tentativa dentro da SisFall (azul; validação por sujeito) e na MobiFall sem retreino (laranja). À direita, a perda ΔF1 com IC 95% (bootstrap de sujeitos). Pontos azuis são limiares; pontos cinza escuro são variantes da CNN-LSTM.*

| Método | SisFall SE | SisFall SP | SisFall F1 | MobiFall SE | MobiFall SP | MobiFall F1 | MobiFall FA/h | ΔF1 [IC 95%] |
|---|---|---|---|---|---|---|---|---|
| Kangas (publicado) | 93,5% | 97,4% | 0,947 | 89,6% | 90,4% | 0,891 | 13,1 | 0,06 [0,01; 0,10] |
| Bourke3 (publicado) | 60,9% | 99,7% | 0,755 | 9,0% | 98,5% | 0,163 | 5,2 | 0,59 [0,51; 0,68] |
| PIPTO (publicado) | 81,3% | 88,2% | 0,816 | 4,5% | 100,0% | 0,086 | 0,0 | 0,73 [0,67; 0,79] |
| Kangas (ajustado) | 96,7% | 97,4% | 0,964 | 96,9% | 89,2% | 0,924 | 14,7 | 0,04 [0,00; 0,08] |
| Bourke3 (ajustado) | 63,0% | 99,4% | 0,768 | 34,0% | 94,7% | 0,485 | 11,6 | 0,28 [0,19; 0,39] |
| CNN-LSTM [z treino] k=2 | 87,8% | 99,8% | 0,934 | 15,6% | 72,8% | 0,211 | 37,1 | 0,72 [0,64; 0,80] |
| CNN-LSTM [z treino] k=1 | 99,4% | 99,0% | 0,989 | 54,5% | 67,3% | 0,564 | 44,6 | 0,43 [0,34; 0,50] |
| CNN-LSTM [z por base] k=2 | 87,8% | 99,8% | 0,934 | 0,0% | 63,2% | 0,000 | 50,2 | 0,93 [0,92; 0,95] |
| CNN-LSTM [z por base] k=1 | 99,4% | 99,0% | 0,989 | 1,4% | 52,9% | 0,018 | 64,2 | 0,97 [0,95; 0,99] |
| CNN-LSTM rot [z treino] k=2 | 89,4% | 99,9% | 0,943 | 15,3% | 99,4% | 0,263 | 0,8 | 0,68 [0,59; 0,78] |
| CNN-LSTM rot [z treino] k=1 | 99,4% | 98,4% | 0,985 | 67,4% | 94,7% | 0,776 | 7,2 | 0,21 [0,16; 0,25] |
| CNN-LSTM rot [z por base] k=2 | 89,4% | 99,9% | 0,943 | 4,9% | 100,0% | 0,093 | 0,0 | 0,85 [0,78; 0,90] |
| CNN-LSTM rot [z por base] k=1 | 99,4% | 98,4% | 0,985 | 46,9% | 99,4% | 0,635 | 0,8 | 0,35 [0,30; 0,41] |

*Tabela: SisFall → MobiFall. "z treino": normalização com as estatísticas da base de treino (justa). "z por base": cada base normalizada com as próprias estatísticas (protocolo do artigo). "rot": treinada com rotações 3D. k: nº de janelas positivas seguidas para declarar queda.*

## 5.3 Do bolso para a cintura: MobiFall → SisFall

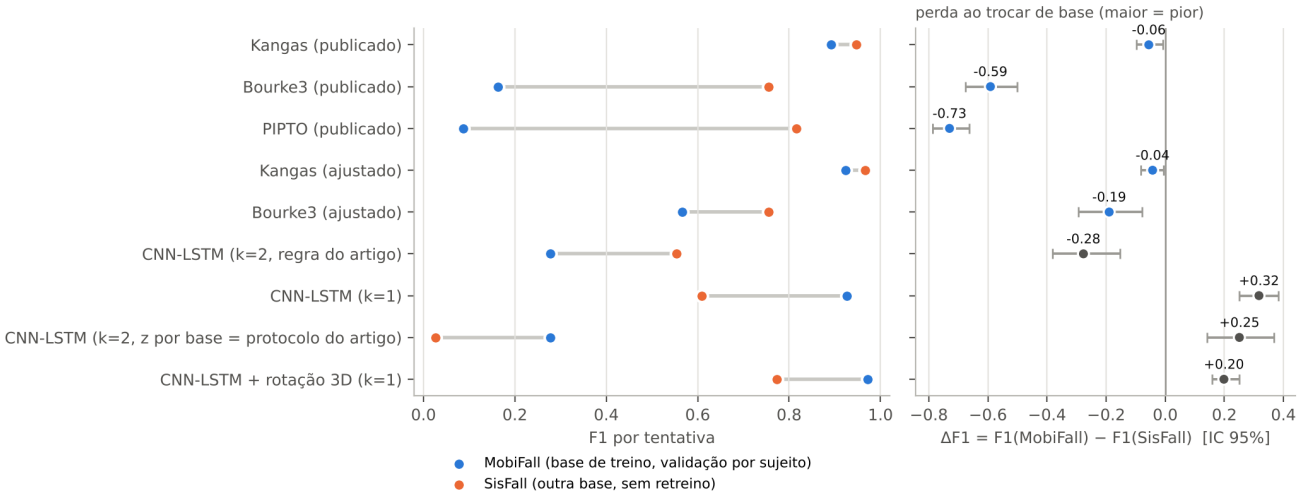



*Figura 2. O mesmo que a Figura 1, na direção inversa. Δ negativo significa que o método foi melhor na base nova do que na de treino.*

| Método | MobiFall SE | MobiFall SP | MobiFall F1 | SisFall SE | SisFall SP | SisFall F1 | SisFall FA/h | ΔF1 [IC 95%] |
|---|---|---|---|---|---|---|---|---|
| Kangas (publicado) | 89,6% | 90,4% | 0,891 | 93,5% | 97,4% | 0,947 | 4,8 | -0,06 [-0,10; -0,01] |
| Bourke3 (publicado) | 9,0% | 98,5% | 0,163 | 60,9% | 99,7% | 0,755 | 0,6 | -0,59 [-0,68; -0,50] |
| PIPTO (publicado) | 4,5% | 100,0% | 0,086 | 81,3% | 88,2% | 0,816 | 21,9 | -0,73 [-0,79; -0,66] |
| Kangas (ajustado) | 96,9% | 89,2% | 0,924 | 98,0% | 96,9% | 0,967 | 5,8 | -0,04 [-0,08; -0,01] |
| Bourke3 (ajustado) | 42,4% | 93,9% | 0,566 | 61,4% | 99,3% | 0,756 | 1,5 | -0,19 [-0,29; -0,08] |
| CNN-LSTM [z treino] k=2 | 16,3% | 98,8% | 0,277 | 73,0% | 40,4% | 0,554 | 111,1 | -0,28 [-0,38; -0,15] |
| CNN-LSTM [z treino] k=1 | 93,8% | 92,7% | 0,926 | 96,8% | 20,2% | 0,609 | 148,8 | 0,32 [0,25; 0,38] |
| CNN-LSTM [z por base] k=2 | 16,3% | 98,8% | 0,277 | 1,4% | 95,4% | 0,026 | 8,6 | 0,25 [0,14; 0,37] |
| CNN-LSTM [z por base] k=1 | 93,8% | 92,7% | 0,926 | 8,5% | 75,8% | 0,118 | 45,1 | 0,81 [0,72; 0,88] |
| CNN-LSTM rot [z treino] k=2 | 33,3% | 100,0% | 0,500 | 32,5% | 90,4% | 0,442 | 17,8 | 0,06 [-0,05; 0,16] |
| CNN-LSTM rot [z treino] k=1 | 95,8% | 98,8% | 0,972 | 89,1% | 72,8% | 0,773 | 50,8 | 0,20 [0,16; 0,25] |
| CNN-LSTM rot [z por base] k=2 | 33,3% | 100,0% | 0,500 | 27,1% | 93,6% | 0,396 | 12,0 | 0,10 [-0,01; 0,21] |
| CNN-LSTM rot [z por base] k=1 | 95,8% | 98,8% | 0,972 | 84,2% | 75,1% | 0,759 | 46,4 | 0,21 [0,17; 0,26] |

Nesta direção, a CNN-LSTM treinada no bolso (k=1) mantém a sensibilidade na SisFall (96,8%), mas a especificidade despenca para 20,2%: são 149 alarmes falsos por hora. Andar, correr e subir escada viram "queda". O Kangas reajustado no bolso funciona ainda melhor na cintura: F1 0,967.

## 5.4 Por que Bourke3 e PIPTO caíram no celular: a faixa do sensor

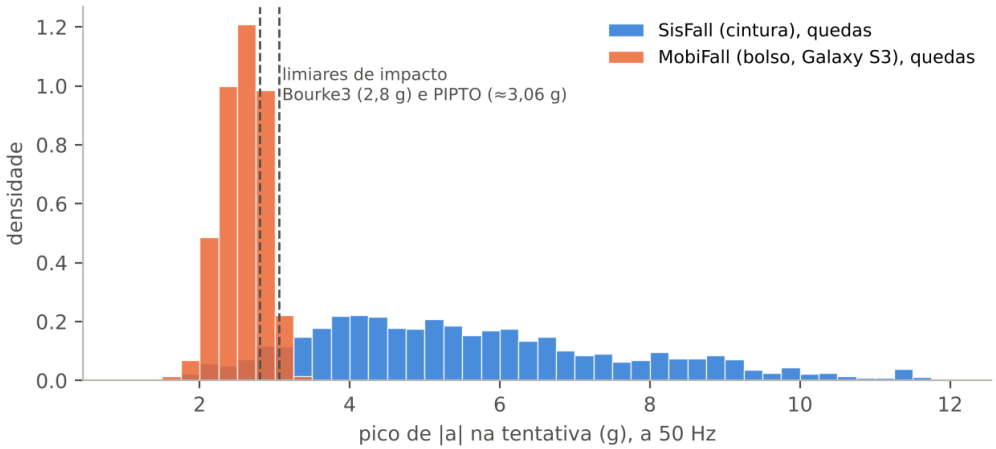



*Figura 3. Pico de |a| em cada tentativa de queda. Na MobiFall, nenhuma queda passa de 3,35 g, porque o acelerômetro do Galaxy S3 satura em ±2 g por eixo (máximo de 19,6 m/s² medido nos arquivos brutos). Na SisFall, o pico mediano das quedas é de 5,3 g.*

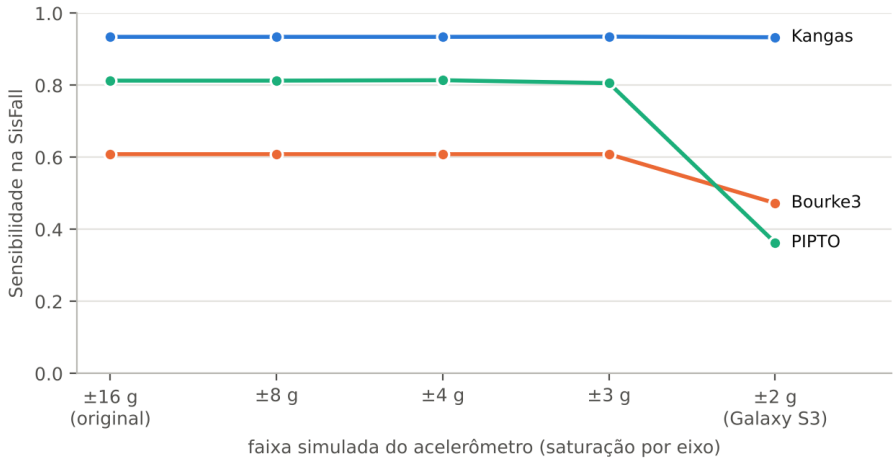



*Figura 4. Controle: sensibilidade dos limiares publicados na própria SisFall, cortando o sinal em faixas cada vez menores. Kangas, que não depende de um pico alto, não muda. PIPTO e Bourke3 caem quando a faixa chega a ±2 g.*

| Faixa simulada | Kangas SE / SP | Bourke3 SE / SP | PIPTO SE / SP |
|---|---|---|---|
| sem limite (±16 g do sensor) | 93,5% / 97,4% | 60,9% / 99,7% | 81,3% / 88,2% |
| ±8 g | 93,5% / 97,4% | 60,9% / 99,7% | 81,3% / 88,2% |
| ±4 g | 93,5% / 97,4% | 60,9% / 99,7% | 81,4% / 88,3% |
| ±3 g | 93,5% / 97,4% | 60,9% / 99,7% | 80,6% / 88,5% |
| ±2 g | 93,3% / 97,4% | 47,3% / 100,0% | 36,3% / 98,2% |

A saturação explica boa parte, mas não tudo. Na MobiFall a queda foi maior que na SisFall saturada (PIPTO: 4,5% contra 36%). O resto vem da **posição**: no bolso o impacto chega amortecido pela coxa, e as quedas da MobiFall foram feitas com o celular no bolso do lado oposto ao da queda.

## 5.5 Por que a CNN-LSTM caiu: orientação, normalização e instabilidade

- **Orientação.** Com a pessoa em pé, a gravidade aparece em −y na SisFall e em +y na MobiFall: o sensor está "de cabeça para baixo" de uma base para outra. A CNN usa ax, ay e az crus e nunca viu sinais assim no treino. Treinar com rotações 3D aleatórias reduziu a perda de 0,43 para 0,21 (k=1) e subiu a especificidade na MobiFall de 67,3% para 94,7%.
- **Normalização do artigo ("z por base").** Normalizar cada base com as próprias estatísticas, como fez Duan et al., piorou a transferência (F1 na MobiFall de 0,000). Isso mostra que o resultado de ≈1% de perda do artigo depende de as duas bases serem parecidas.
- **Instabilidade entre sementes.** Treinando a mesma rede 3 vezes, só mudando a semente aleatória, o F1 na MobiFall (k=1) variou de 0,44 a 0,77. Dentro da base essa variação é mínima (desvio da acurácia por janela de 0,3%). Um único treino pode dar uma impressão muito otimista ou pessimista.
- **Regra k=2 do artigo.** Exigir 2 janelas positivas seguidas reduz alarmes falsos dentro da base, mas fora dela derruba a sensibilidade (MobiFall: 15,6%).

| Direção | Variante | Acurácia por janela, dentro | Recall por janela, dentro | Acurácia por janela, outra base | Recall, outra base | Precisão, outra base |
|---|---|---|---|---|---|---|
| SisFall→MobiFall | CNN-LSTM | 99,0% ± 0,3% | 93,0% | 69,8% | 50,6% | 14,6% |
| SisFall→MobiFall | CNN-LSTM rot | 99,1% ± 0,2% | 93,7% | 95,0% | 56,9% | 79,9% |
| MobiFall→SisFall | CNN-LSTM | 95,7% ± 3,0% | 73,4% | 51,1% | 68,2% | 14,4% |
| MobiFall→SisFall | CNN-LSTM rot | 98,0% ± 0,9% | 85,8% | 91,8% | 60,6% | 61,8% |

*Métricas por janela, como as do artigo. Dentro da SisFall, a acurácia por janela (99,0%) reproduz os 99,3% de Duan et al. na KFall.*

## 5.6 Onde cada método erra

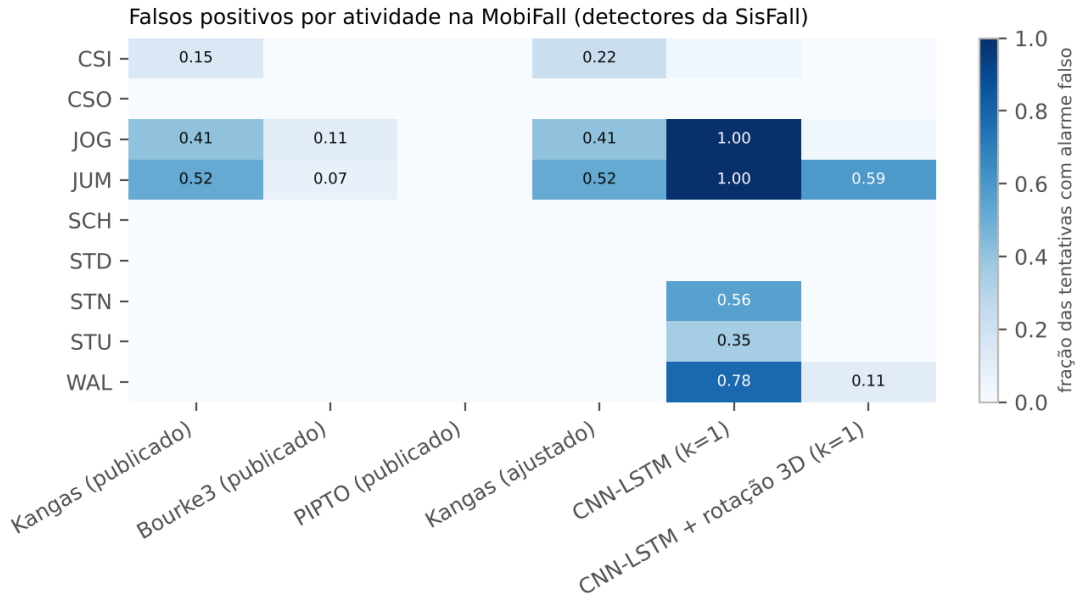



*Figura 5. Fração das tentativas de cada ADL da MobiFall com alarme falso (detectores ajustados ou treinados na SisFall). O Kangas erra em correr (JOG) e pular (JUM). A CNN sem rotação erra em quase tudo que envolve movimento: andar, escada, correr, pular.*

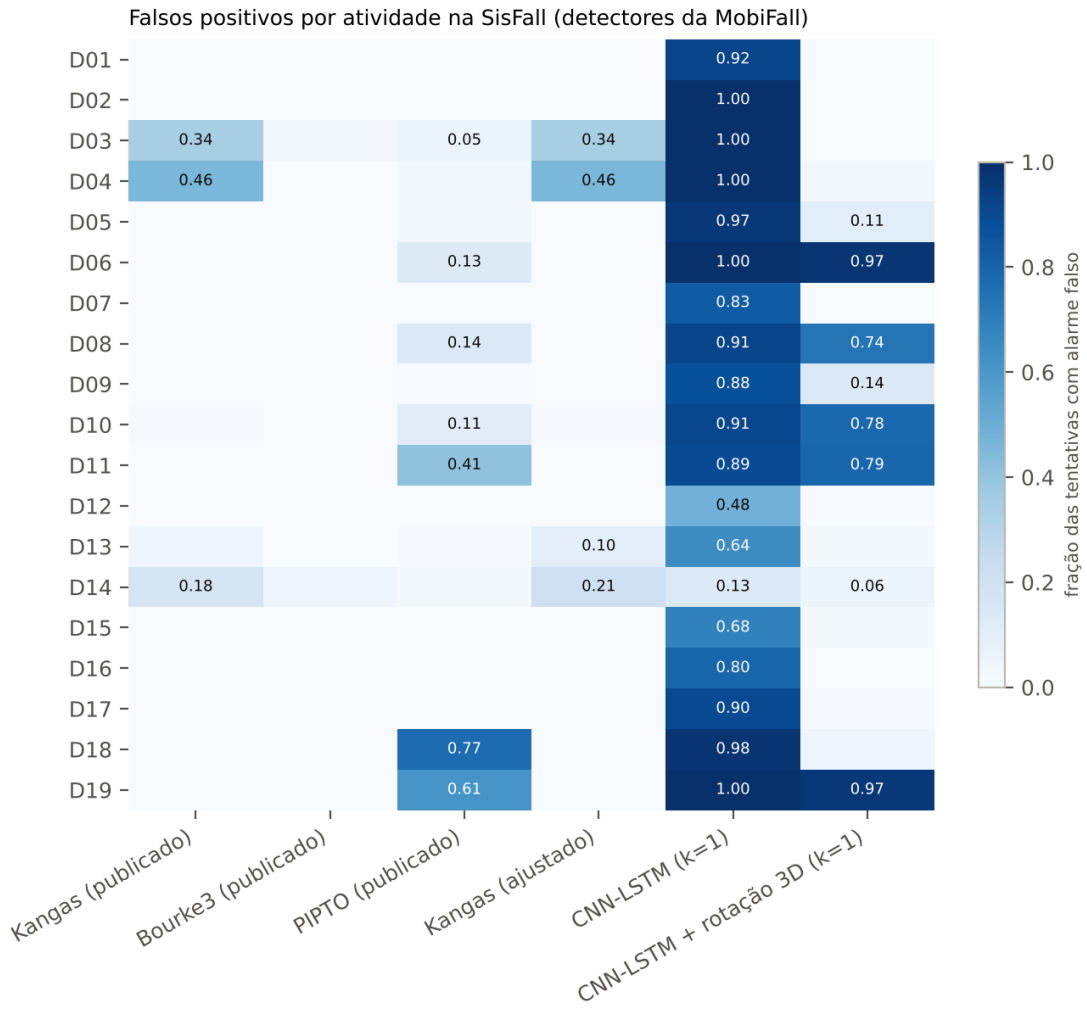



*Figura 6. O mesmo na SisFall, com detectores ajustados ou treinados na MobiFall. As quase-quedas D11 (desaba na cadeira) e D18 (tropeço) enganam o PIPTO; correr (D03–D04) engana o Kangas.*

| Método (ajustado/treinado na MobiFall) | SP na SisFall, jovens | SP na SisFall, idosos |
|---|---|---|
| Kangas (publicado) | 97,9% | 96,4% |
| Kangas (ajustado) | 97,2% | 96,2% |

| Método (ajustado/treinado na MobiFall) | SP na SisFall, jovens | SP na SisFall, idosos |
|---|---|---|
| Bourke3 (publicado) | 99,6% | 99,8% |
| PIPTO (publicado) | 84,0% | 96,4% |
| CNN-LSTM [z treino] k=1 | 15,8% | 28,8% |
| CNN-LSTM rot [z treino] k=1 | 68,8% | 80,5% |

Para Kangas e Bourke3, a especificidade com idosos e com jovens foi praticamente igual (diferença de até 1,5 ponto percentual). Para PIPTO e CNN, as atividades dos idosos geraram *menos* alarmes falsos, o que é coerente com movimentos mais lentos. Lembrete: na SisFall os idosos não caíram, então não dá para medir a sensibilidade em idosos.

## 5.7 Checagens de validade

| Direção | Método | F1 treino (grade original) | F1 treino (grade ampliada) | F1 outra base (original) | F1 outra base (ampliada) |
|---|---|---|---|---|---|
| SisFall→MobiFall | Kangas (ajustado) | 0,964 | 0,966 | 0,924 | 0,929 |
| SisFall→MobiFall | Bourke3 (ajustado) | 0,768 | 0,765 | 0,485 | 0,481 |
| MobiFall→SisFall | Kangas (ajustado) | 0,924 | 0,918 | 0,967 | 0,968 |
| MobiFall→SisFall | Bourke3 (ajustado) | 0,566 | 0,539 | 0,756 | 0,757 |

Ampliar a grade de limiares não muda nenhuma conclusão: as diferenças são de no máximo 0,03 no F1.

| Algoritmo (SisFall, publicado) | a 50 Hz: SE / SP / F1 | a 200 Hz (original): SE / SP / F1 |
|---|---|---|
| Kangas | 93,5% / 97,4% / 0,947 | 96,0% / 97,0% / 0,957 |
| Bourke3 | 60,9% / 99,7% / 0,755 | 57,7% / 99,6% / 0,729 |
| PIPTO | 81,3% / 88,2% / 0,816 | 81,2% / 86,9% / 0,808 |

Reduzir de 200 para 50 Hz quase não muda os limiares: a diferença no F1 é de no máximo 0,03, com o Kangas um pouco melhor e o Bourke3 um pouco pior a 200 Hz. Portanto, a taxa de 50 Hz usada no estudo não explica nenhuma das conclusões, nem a distância entre o PIPTO aqui (F1 ≈ 0,81) e a acurácia acima de 97% relatada pelos autores na SisFall.

- **Reprodução da CNN:** acurácia por janela de 99,0% dentro da SisFall, contra 99,3% de Duan et al. na KFall. A reimplementação se comporta como a original dentro da base.
- **Unidades:** |a| ≈ 1 g em repouso nas duas bases. O pico do rótulo e o pico do sinal foram conferidos no modo de teste do notebook.
- **Código do PIPTO:** roda sem modificação e sem erros internos em 5.026 tentativas.

# 6. Discussão

## 6.1 O que os resultados dizem sobre as hipóteses

- **H1 é confirmada para o Kangas.** A CNN-LSTM perde de 0,43 a 0,72 de F1 da cintura para o bolso, e o Kangas perde 0,04 a 0,06. Os intervalos de confiança não se sobrepõem.
- **H1 é refutada para Bourke3 e PIPTO.** Eles perdem tanto ou mais que a CNN (ΔF1 de 0,59 e 0,73). A regra de decisão deles depende de um pico de impacto alto, e esse pico é uma característica do *sensor* (faixa, posição), não da queda em si.
- **A H1-alternativa (perda de ≈1%, Duan et al.) não se reproduz** quando a troca de base envolve mudar a posição e o aparelho. Não dá para afirmar o mesmo para KFall → SisFall, em que as duas bases são de cintura ou lombar e com quedas parecidas. Esse teste exige obter a KFall.

## 6.2 Interpretação

Um algoritmo de detecção de quedas generaliza bem quando a regra de decisão se apoia em características da **queda** e não do **sensor**. O Kangas combina quatro sinais diferentes de impacto (basta um) com a queda livre e a postura, e por isso sobrevive à saturação e à troca de posição. A CNN aprende detalhes da base de treino: orientação dos eixos, intensidade típica dos movimentos, até o jeito de cada voluntário cair. Esses detalhes não se transferem. Treinar com rotações ajuda, mas não resolve.

Para o Mova, isso tem duas consequências práticas. (1) Um modelo de IA treinado numa base pública **não deve** ser levado direto para o app sem dados do próprio celular e posição; ele pode gerar dezenas de alarmes falsos por hora. (2) Um limiar simples e bem escolhido, como o Kangas, é uma linha de base forte, mas ainda gera 5 a 15 alarmes falsos por hora em atividades como correr e pular. Para idosos, que raramente correm, esse número deve ser menor, mas isso precisa ser medido.

## 6.3 Comparação com a literatura

- Bagalà et al. (2012) mostraram que limiares perdem muito em quedas *reais* (SE média de 57%). Aqui mostramos que a troca de base com quedas *simuladas* já separa os métodos robustos dos frágeis.
- O PIPTO relata acurácia acima de 97% na SisFall (dado visto só em resumo de busca, a conferir). Aqui ele teve 85,5% de acurácia na SisFall a 50 Hz. A diferença pode vir da regra de pontuação, da forma de separar as tentativas (a taxa de amostragem foi descartada como causa na Seção 5.7) e de o PIPTO ter sido desenvolvido com a KFall e a MMsys.
- Revisões de 2024–2026 apontam a validação "na mesma base" como o principal problema da área. Este estudo mede esse problema de forma direta e reprodutível.

# 7. Limitações e ameaças à validade

- **Quedas simuladas por adultos jovens** nas duas bases. Não há quedas reais de idosos; a SisFall tem só ADLs de idosos.
- **A KFall não foi usada**, então a comparação direta com o protocolo de Duan et al. (KFall → SisFall) continua pendente.
- **Bases obtidas por espelhos sem licença declarada.** O espelho da SisFall tem 4.398 dos 4.505 arquivos. Citem os artigos originais.
- **A MobiFall tem poucos sujeitos com ADL** (9) e o celular dela é antigo (±2 g). Celulares atuais costumam medir ±8 g ou mais, e o resultado do Bourke3 e do PIPTO deve melhorar com eles. Isso precisa de dados novos.
- **Intervalo da queda aproximado pelo pico** (pico − 1 s, pico + 0,5 s) para treinar a CNN, porque SisFall e MobiFall não marcam o instante da queda.
- **Validação cruzada em 5 partes em vez de LOSO**, por tempo de processamento. A regra de separar sujeitos foi mantida.

- **Parâmetros de Kangas e Bourke3** vieram do Bagalà 2012 e do código "Guardian", não dos PDFs originais. Conferir antes de publicar.
- **CNN reimplementada sem o código dos autores.** Lote e ativação são suposições, e o núcleo da 1ª convolução foi corrigido.
- **Pontuação por tentativa:** um alarme em qualquer ponto de uma tentativa de queda conta como acerto, mesmo fora do momento exato da queda.

## 8. Decisões que a equipe precisa tomar

1. **Título e recorte do artigo.** Sugestão: "Limiares ou aprendizado de máquina? Generalização entre bases na detecção de quedas com sensores inerciais, da cintura ao smartphone".
2. **Pedir acesso à KFall** e rodar a direção KFall → SisFall. Basta acrescentar a direção no notebook, cerca de 4 h de CPU ou bem menos com GPU no Colab.
3. **Conferir no PDF** os parâmetros de Kangas (Gait & Posture 2008) e Bourke3 (J. Biomech. 2010) e os números do PIPTO.
4. **Opcional, para o app:** medir a faixa do acelerômetro dos celulares da equipe (expo-sensors) e gravar algumas ADLs no bolso para estimar alarmes falsos por hora.

## 9. Reprodutibilidade

- Notebook: `notebooks/estudo_limiar_vs_ml.ipynb`. Com `MODO_DEMO = True`, roda em ≈5 min com dados sintéticos; com `MODO_DEMO = False`, usa as bases reais (Seção 2 do notebook).
- Resultados brutos (CSV): `docs/resultados_experimento/`. Inclui previsões por tentativa de todos os métodos, tabelas, erros por atividade, controles de saturação e de grade e a variação entre sementes.
- Sementes fixas (42) e versões das bibliotecas registradas. Mesmo assim, os resultados da CNN variam com a semente, como mostra a Seção 5.5.

## Referências


Bagalà F. et al. Evaluation of accelerometer-based fall detection algorithms on real-world falls. PLoS ONE 7(5):e37062, 2012.

Bourke A. K. et al. Evaluation of waist-mounted tri-axial accelerometer based fall-detection algorithms during scripted and continuous unscripted activities. J. Biomechanics 43(15):3051–3057, 2010.

Duan L.-C., Minh N.-N., Dung T.-C., Linh T.-T.-T. Energy-efficient TinyML approach for wearable fall detection on edge devices using spatial-temporal deep learning. Int. J. Technology 17(3):919–935, 2026. doi:10.14716/ijtech.v17i3.8445.

Kangas M. et al. Comparison of low-complexity fall detection algorithms for body attached accelerometers. Gait & Posture 28(2):285–291, 2008.

Moutsis S. N., Tsintotas K. A., Gasteratos A. PIPTO: Precise inertial-based pipeline for threshold-based fall detection using three-axis accelerometers. Sensors 23(18):7951, 2023.

Sucerquia A., López J. D., Vargas-Bonilla J. F. SisFall: a fall and movement dataset. Sensors 17(1):198, 2017.

Vavoulas G. et al. The MobiFall dataset: fall detection and classification with a smartphone. Int. J. Monitoring and Surveillance Technologies Research 2(1), 2014.

Yu X., Jang J., Xiong S. A large-scale open motion dataset (KFall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors. Front. Aging Neurosci. 13:692865, 2021.